

Java-Referenz

Benjamin Malicke ©

November 2, 2025

Contents

1	Einführung	2
2	Java-Schlüsselwörter	3
2.1	Beispielcode	4
3	Java String-Methoden	5
3.1	Beispielcode	6
4	Java Math-Methoden	7
4.1	Beispielcode	8
5	Java Ausgabe-Methoden	9
5.1	Beispielcode	9
6	Java Arrays-Methoden	10
6.1	Beispielcode	10
7	Java ArrayList-Methoden	11
7.1	Beispielcode	11
8	Java LinkedList-Methoden	12
8.1	Beispielcode	13
9	Java HashMap-Methoden	14
10	Java Scanner-Methoden	15
10.1	Beispielcode	15
11	Java File-Methoden	16
11.1	Beispielcode	16
12	Java FileInputStream-Methoden	17
12.1	Beispielcode	17
13	Java FileOutputStream-Methoden	18
13.1	Beispielcode	18
14	Java BufferedReader-Methoden	19
14.1	Beispielcode	19
15	Java BufferedWriter-Methoden	20
15.1	Beispielcode	20
16	Java Iterator-Methoden	21
16.1	Beispielcode	21

17 Java Collections-Methoden	22
17.1 Beispielcode	22
18 Java System-Methoden	23
18.1 Beispielcode	23
19 Java Fehler und Ausnahmen	24
19.1 Beispielcode	24
20 Java-Datum	24

1 Einführung

Diese Referenz bietet eine Übersicht über die reservierten Schlüsselwörter in Java sowie die wichtigsten Methoden der Klassen `String`, `Math`, `Arrays`, `ArrayList`, `LinkedList`, `HashMap`, `Scanner`, `File`, `FileInputStream`, `FileOutputStream`, `BufferedReader`, `BufferedWriter`, `Iterator`, `Collections`, `System` und häufige Fehler und Ausnahmen. Die Tabellen listen die Schlüsselwörter und Methoden mit ihren Beschreibungen auf. Beispielcode demonstriert die Anwendung ausgewählter Java-Elemente.

2 Java-Schlüsselwörter

Schlüsselwort	Beschreibung
<code>abstract</code>	Nicht-Zugriffsmodifikator. Wird für Klassen und Methoden verwendet: Eine abstrakte Klasse kann keine Objekte erzeugen (muss von einer anderen Klasse geerbt werden). Eine abstrakte Methode kann nur in einer abstrakten Klasse verwendet werden und hat keinen Rumpf. Der Rumpf wird von der Unterklasse bereitgestellt.
<code>assert</code>	Für Debugging-Zwecke.
<code>boolean</code>	Datentyp, der nur <code>true</code> oder <code>false</code> speichern kann.
<code>break</code>	Bricht aus einer Schleife oder einem Switch-Block aus.
<code>byte</code>	Datentyp, der Ganzzahlen von -128 bis 127 speichern kann.
<code>case</code>	Markiert einen Codeblock in einem Switch-Statement.
<code>catch</code>	Fängt Ausnahmen ab, die von Try-Statements generiert werden.
<code>char</code>	Datentyp, der ein einzelnes Zeichen speichert.
<code>class</code>	Definiert eine Klasse.
<code>continue</code>	Führt mit der nächsten Iteration einer Schleife fort.
<code>const</code>	Definiert eine Konstante. Nicht in Gebrauch - stattdessen <code>final</code> verwenden.
<code>default</code>	Gibt den Standard-Codeblock in einem Switch-Statement an.
<code>do</code>	Wird zusammen mit <code>while</code> verwendet, um eine Do-While-Schleife zu erstellen.
<code>double</code>	Datentyp, der Gleitkommazahlen von $1.7 \cdot 10^{-308}$ bis $1.7 \cdot 10^{308}$ speichern kann.
<code>else</code>	Wird in bedingten Anweisungen verwendet.
<code>enum</code>	Deklariert einen unveränderlichen (aufzählenden) Typ.
<code>exports</code>	Exportiert ein Paket mit einem Modul. Neu in Java 9.
<code>extends</code>	Erweitert eine Klasse (zeigt an, dass eine Klasse von einer anderen geerbt wird).
<code>final</code>	Nicht-Zugriffsmodifikator für Klassen, Attribute und Methoden, der sie unveränderbar macht (Vererbung oder Überschreibung unmöglich).
<code>finally</code>	Wird mit Ausnahmen verwendet, ein Codeblock, der immer ausgeführt wird, unabhängig von einer Ausnahme.
<code>float</code>	Datentyp, der Gleitkommazahlen von $3.4 \cdot 10^{-38}$ bis $3.4 \cdot 10^{38}$ speichert.
<code>for</code>	Erstellt eine For-Schleife.
<code>goto</code>	Nicht in Gebrauch, hat keine Funktion.
<code>if</code>	Erstellt eine bedingte Anweisung.
<code>implements</code>	Implementiert ein Interface.
<code>import</code>	Importiert ein Paket, eine Klasse oder ein Interface.
<code>instanceof</code>	Prüft, ob ein Objekt eine Instanz einer bestimmten Klasse oder eines Interfaces ist.
<code>int</code>	Datentyp, der Ganzzahlen von -2147483648 bis 2147483647 speichert.
<code>interface</code>	Deklariert einen speziellen Klassentyp, der nur abstrakte Methoden enthält.
<code>long</code>	Datentyp, der Ganzzahlen von -9223372036854775808 bis 9223372036854775808 speichert.
<code>module</code>	Deklariert ein Modul. Neu in Java 9.
<code>native</code>	Gibt an, dass eine Methode nicht in derselben Java-Quelldatei implementiert ist (sondern in einer anderen Sprache).
<code>new</code>	Erstellt neue Objekte.
<code>package</code>	Deklariert ein Paket.
<code>private</code>	Zugriffsmodifikator für Attribute, Methoden und Konstruktoren, die nur innerhalb der deklarierten Klasse zugänglich sind.
<code>protected</code>	Zugriffsmodifikator für Attribute, Methoden und Konstruktoren, die im selben Paket und in Unterklassen zugänglich sind.
<code>public</code>	Zugriffsmodifikator für Klassen, Attribute, Methoden und Konstruktoren, die von jeder anderen Klasse zugänglich sind.
<code>requires</code>	Gibt erforderliche Bibliotheken in einem Modul an. Neu in Java 9.
<code>return</code>	Beendet die Ausführung einer Methode und kann einen Wert zurückgeben.

Schlüsselwort	Beschreibung
short	Datentyp, der Ganzzahlen von -32768 bis 32767 speichert.
static	Nicht-Zugriffsmodifikator für Methoden und Attribute. Statische Methoden/Attribute können ohne Erstellung eines Klassenobjekts aufgerufen werden.
strictfp	Veraltet. Beschränkt die Präzision und Rundung von Gleitkommaberechnungen.
super	Verweist auf Objekte der Superklasse (Elternklasse).
switch	Wählt einen von vielen Codeblöcken zur Ausführung aus.
synchronized	Nicht-Zugriffsmodifikator, der angibt, dass Methoden nur von einem Thread gleichzeitig aufgerufen werden können.
this	Verweist auf das aktuelle Objekt in einer Methode oder einem Konstruktor.
throw	Erstellt einen benutzerdefinierten Fehler.
throws	Gibt an, welche Ausnahmen eine Methode werfen kann.
transient	Wird verwendet, um ein Attribut bei der Serialisierung eines Objekts zu ignorieren.
try	Erstellt eine Try-Catch-Anweisung.
var	Deklariert eine Variable. Neu in Java 10.
void	Gibt an, dass eine Methode keinen Rückgabewert hat.
volatile	Gibt an, dass ein Attribut nicht lokal im Thread gecacht wird und immer aus dem „Hauptspeicher“ gelesen wird.
while	Erstellt eine While-Schleife.

2.1 Beispielcode

```

1 public class Example {
2     public static void main(String[] args) {
3         int x = 10;
4         if (x > 5) {
5             System.out.println("x ist groesser als 5");
6         } else {
7             System.out.println("x ist kleiner oder gleich 5");
8         }
9     }
10 }

```

3 Java String-Methoden

Methoden	Beschreibung	Rückgabotyp
<code>charAt()</code>	Gibt das Zeichen an der angegebenen Position (Index) zurück.	<code>char</code>
<code>codePointAt()</code>	Gibt den Unicode-Wert des Zeichens an der angegebenen Position zurück.	<code>int</code>
<code>codePointBefore()</code>	Gibt den Unicode-Wert des Zeichens vor der angegebenen Position zurück.	<code>int</code>
<code>codePointCount()</code>	Gibt die Anzahl der Unicode-Werte in einem String zurück.	<code>int</code>
<code>compareTo()</code>	Vergleicht zwei Strings lexikografisch.	<code>int</code>
<code>compareToIgnoreCase()</code>	Vergleicht zwei Strings lexikografisch, ohne Groß-/Kleinschreibung zu berücksichtigen.	<code>int</code>
<code>concat()</code>	Fügt einen String an das Ende eines anderen Strings an.	<code>String</code>
<code>contains()</code>	Prüft, ob ein String eine Zeichenfolge enthält.	<code>boolean</code>
<code>contentEquals()</code>	Prüft, ob ein String exakt dieselbe Zeichenfolge wie eine angegebene <code>CharSequence</code> oder <code>StringBuffer</code> enthält.	<code>boolean</code>
<code>copyValueOf()</code>	Gibt einen String zurück, der die Zeichen eines Zeichenarrays darstellt.	<code>String</code>
<code>endsWith()</code>	Prüft, ob ein String mit den angegebenen Zeichen endet.	<code>boolean</code>
<code>equals()</code>	Vergleicht zwei Strings. Gibt <code>true</code> zurück, wenn die Strings gleich sind, sonst <code>false</code> .	<code>boolean</code>
<code>equalsIgnoreCase()</code>	Vergleicht zwei Strings, ohne Groß-/Kleinschreibung zu berücksichtigen.	<code>boolean</code>
<code>format()</code>	Gibt einen formatierten String mit der angegebenen Lokalisierung, Formatzeichenkette und Argumenten zurück.	<code>String</code>
<code>getBytes()</code>	Konvertiert einen String in ein Byte-Array.	<code>byte[]</code>
<code>getChars()</code>	Kopiert Zeichen aus einem String in ein Zeichen-Array.	<code>void</code>
<code>hashCode()</code>	Gibt den Hash-Code eines Strings zurück.	<code>int</code>
<code>indexOf()</code>	Gibt die Position des ersten Auftretens angegebener Zeichen in einem String zurück.	<code>int</code>
<code>intern()</code>	Gibt die kanonische Darstellung des String-Objekts zurück.	<code>String</code>
<code>isEmpty()</code>	Prüft, ob ein String leer ist.	<code>boolean</code>
<code>join()</code>	Verbindet einen oder mehrere Strings mit einem angegebenen Trennzeichen.	<code>String</code>
<code>lastIndexOf()</code>	Gibt die Position des letzten Auftretens angegebener Zeichen in einem String zurück.	<code>int</code>
<code>length()</code>	Gibt die Länge eines Strings zurück.	<code>int</code>
<code>matches()</code>	Sucht in einem String nach einem Treffer mit einem regulären Ausdruck und gibt die Treffer zurück.	<code>boolean</code>
<code>offsetByCodePoints()</code>	Gibt den Index innerhalb dieses Strings zurück, der um <code>codePointOffset</code> Codepunkte vom angegebenen Index verschoben ist.	<code>int</code>
<code>regionMatches()</code>	Prüft, ob zwei String-Bereiche gleich sind.	<code>boolean</code>
<code>replace()</code>	Sucht in einem String nach einem angegebenen Wert und gibt einen neuen String zurück, bei dem die angegebenen Werte ersetzt wurden.	<code>String</code>
<code>replaceAll()</code>	Ersetzt jede Teilzeichenfolge dieses Strings, die mit dem angegebenen regulären Ausdruck übereinstimmt, durch den angegebenen Ersatz.	<code>String</code>
<code>replaceFirst()</code>	Ersetzt das erste Auftreten einer Teilzeichenfolge, die mit dem angegebenen regulären Ausdruck übereinstimmt, durch den angegebenen Ersatz.	<code>String</code>
<code>split()</code>	Teilt einen String in ein Array von Teilstrings.	<code>String[]</code>
<code>startsWith()</code>	Prüft, ob ein String mit den angegebenen Zeichen beginnt.	<code>boolean</code>

Methode	Beschreibung	Rückgabebetyp
subSequence()	Gibt eine neue Zeichenfolge zurück, die eine Teilsequenz dieser Sequenz ist.	CharSequence
substring()	Gibt einen neuen String zurück, der eine Teilzeichenfolge eines angegebenen Strings ist.	String
toCharArray()	Konvertiert diesen String in ein neues Zeichen-Array.	char[]
toLowerCase()	Konvertiert einen String in Kleinbuchstaben.	String
toString()	Gibt den Wert eines String-Objekts zurück.	String
toUpperCase()	Konvertiert einen String in Großbuchstaben.	String
trim()	Entfernt Leerzeichen von beiden Enden eines Strings.	String
valueOf()	Gibt die String-Darstellung des angegebenen Werts zurück.	String

3.1 Beispielcode

```

1 public class StringExample {
2     public static void main(String[] args) {
3         String text = "Hallo Welt";
4         System.out.println("Laenge: " + text.length());
5         System.out.println("In Grossbuchstaben: " + text.toUpperCase());
6         System.out.println("Teilstring: " + text.substring(0, 5));
7     }
8 }

```

4 Java Math-Methoden

Methode	Beschreibung	Rückgabotyp
abs(x)	Gibt den absoluten Wert von x zurück.	double float int long
acos(x)	Gibt den Arkuskosinus von x in Radiant zurück.	double
addExact(x, y)	Gibt die Summe von x und y zurück.	int long
asin(x)	Gibt den Arkussinus von x in Radiant zurück.	double
atan(x)	Gibt den Arkustangens von x als numerischer Wert zwischen -PI/2 und PI/2 Radiant zurück.	double
atan2(y, x)	Gibt den Winkel theta aus der Umwandlung von kartesischen Koordinaten (x, y) in Polarkoordinaten (r, theta) zurück.	double
cbrt(x)	Gibt die Kubikwurzel von x zurück.	double
ceil(x)	Gibt den Wert von x auf die nächsthöhere Ganzzahl gerundet zurück.	double
copySign(x, y)	Gibt den ersten Gleitkommawert x mit dem Vorzeichen des zweiten Gleitkommawerts y zurück.	double float
cos(x)	Gibt den Kosinus von x (x in Radiant) zurück.	double
cosh(x)	Gibt den hyperbolischen Kosinus eines double-Werts zurück.	double
decrementExact(x)	Gibt x-1 zurück.	int long
exp(x)	Gibt den Wert von e^x zurück.	double
expm1(x)	Gibt $e^x - 1$ zurück.	double
floor(x)	Gibt den Wert von x auf die nächstniedrigere Ganzzahl gerundet zurück.	double
floorDiv(x, y)	Gibt die Division zwischen x und y abgerundet zurück.	int long
floorMod(x, y)	Gibt den Rest einer Division zwischen x und y zurück, wobei das Ergebnis der Division abgerundet wurde.	int long
getExponent(x)	Gibt den unverzerrten Exponenten von x zurück.	int
hypot(x, y)	Gibt $\sqrt{x^2 + y^2}$ ohne Zwischenüberlauf oder -unterlauf zurück.	double
IEEERemainder(x, y)	Berechnet die Restoperation von x und y gemäß dem IEEE 754 Standard.	double
incrementExact(x)	Gibt x+1 zurück.	int double
log(x)	Gibt den natürlichen Logarithmus (Basis e) von x zurück.	double
log10(x)	Gibt den Logarithmus zur Basis 10 von x zurück.	double
log1p(x)	Gibt den natürlichen Logarithmus der Summe von x und 1 zurück.	double
max(x, y)	Gibt den höchsten Wert von x und y zurück.	double float int long
min(x, y)	Gibt den niedrigsten Wert von x und y zurück.	double float int long
multiplyExact(x, y)	Gibt das Ergebnis von x multipliziert mit y zurück.	int long
negateExact(x)	Gibt die Negation von x zurück.	int long
nextAfter(x, y)	Gibt die Gleitkommazahl zurück, die an x in Richtung y angrenzt.	double float
nextDown(x)	Gibt den Gleitkommawert zurück, der an x in negativer Richtung angrenzt.	double float
nextUp(x)	Gibt den Gleitkommawert zurück, der an x in Richtung positiver Unendlichkeit angrenzt.	double float
pow(x, y)	Gibt den Wert von x hoch y zurück.	double
random()	Gibt eine Zufallszahl zwischen 0 und 1 zurück.	double
rint(x)	Gibt den double-Wert zurück, der x am nächsten ist und einer mathematischen Ganzzahl entspricht.	double
round(x)	Gibt den Wert von x auf die nächstliegende Ganzzahl gerundet zurück.	long int
scalb(x, y)	Gibt x multipliziert mit 2 hoch y zurück.	double float
signum(x)	Gibt das Vorzeichen von x zurück.	double float
sin(x)	Gibt den Sinus von x (x in Radiant) zurück.	double
sinh(x)	Gibt den hyperbolischen Sinus eines double-Werts zurück.	double

Methode	Beschreibung	Rückgabebetyp
<code>sqrt(x)</code>	Gibt die Quadratwurzel von <code>x</code> zurück.	<code>double</code>
<code>subtractExact(x, y)</code>	Gibt das Ergebnis von <code>x</code> minus <code>y</code> zurück.	<code>int long</code>
<code>tan(x)</code>	Gibt den Tangens eines Winkels zurück.	<code>double</code>
<code>tanh(x)</code>	Gibt den hyperbolischen Tangens eines <code>double</code> -Werts zurück.	<code>double</code>
<code>toDegrees(x)</code>	Konvertiert einen Winkel in Radiant in einen ungefähren Winkel in Grad.	<code>double</code>
<code>toIntExact(x)</code>	Konvertiert einen <code>long</code> -Wert in einen <code>int</code> -Wert.	<code>int</code>
<code>toRadians(x)</code>	Konvertiert einen Winkel in Grad in einen ungefähren Winkel in Radiant.	<code>double</code>
<code>ulp(x)</code>	Gibt die Größe der Einheit der geringsten Präzision (ulp) von <code>x</code> zurück.	<code>double float</code>

4.1 Beispielcode

```

1 public class MathExample {
2     public static void main(String[] args) {
3         double x = 16.0;
4         System.out.println("Quadratwurzel: " + Math.sqrt(x));
5         System.out.println("Maximum: " + Math.max(10, 20));
6         System.out.println("Zufallszahl: " + Math.random());
7     }
8 }

```


5 Java Ausgabe-Methoden

Methode	Beschreibung
<code>print()</code>	Gibt Text oder Werte auf der Konsole aus.
<code>printf()</code>	Gibt formatierten Text oder Werte auf der Konsole aus.
<code>println()</code>	Gibt Text oder Werte auf der Konsole aus, gefolgt von einer neuen Zeile.

5.1 Beispielcode

```
1 public class OutputExample {  
2     public static void main(String[] args) {  
3         System.out.println("Hallo Welt");  
4         System.out.printf("Zahl: %d, Text: %s%n", 42, "Test");  
5         System.out.print("Ohne neue Zeile");  
6     }  
7 }
```

6 Java Arrays-Methoden

Methode	Beschreibung
<code>compare()</code>	Vergleicht zwei Arrays.
<code>copyOf()</code>	Erstellt eine Kopie eines Arrays mit einer neuen Länge.
<code>deepEquals()</code>	Vergleicht zwei mehrdimensionale Arrays, um zu prüfen, ob sie tief gleich sind.
<code>equals()</code>	Prüft, ob zwei Arrays gleich sind.
<code>fill()</code>	Füllt ein Array mit einem angegebenen Wert.
<code>mismatch()</code>	Gibt die Indexposition des ersten Konflikts zwischen zwei Arrays zurück.
<code>sort()</code>	Sortiert ein Array in aufsteigender Reihenfolge.

6.1 Beispielcode

```
1 import java.util.Arrays;
2
3 public class ArraysExample {
4     public static void main(String[] args) {
5         int[] numbers = {5, 2, 8, 1, 9};
6         Arrays.sort(numbers);
7         System.out.println("Sortiertes Array: " + Arrays.toString(numbers));
8     }
9 }
```

7 Java ArrayList-Methoden

Methode	Beschreibung	Rückgabotyp
<code>add()</code>	Fügt ein Element zur Liste hinzu.	<code>boolean void</code>
<code>addAll()</code>	Fügt eine Sammlung von Elementen zur Liste hinzu.	<code>boolean</code>
<code>clear()</code>	Entfernt alle Elemente aus der Liste.	<code>void</code>
<code>clone()</code>	Erstellt eine Kopie der ArrayList.	<code>Object</code>
<code>contains()</code>	Prüft, ob ein Element in der Liste vorhanden ist.	<code>boolean</code>
<code>ensureCapacity()</code>	Erhöht die Kapazität der Liste, um eine angegebene Anzahl von Elementen aufnehmen zu können.	<code>void</code>
<code>forEach()</code>	Führt eine Aktion für jedes Element in der Liste aus.	<code>void</code>
<code>get()</code>	Gibt das Element an einer bestimmten Position in der Liste zurück.	<code>T</code>
<code>indexOf()</code>	Gibt die Position des ersten Auftretens eines Elements in der Liste zurück.	<code>int</code>
<code>isEmpty()</code>	Prüft, ob die Liste leer ist.	<code>boolean</code>
<code>iterator()</code>	Gibt ein Iterator-Objekt für die ArrayList zurück.	<code>Iterator</code>
<code>lastIndexOf()</code>	Gibt die Position des letzten Auftretens eines Elements in der Liste zurück.	<code>int</code>
<code>listIterator()</code>	Gibt ein ListIterator-Objekt für die ArrayList zurück.	<code>ListIterator</code>
<code>remove()</code>	Entfernt ein Element aus der Liste.	<code>boolean T</code>
<code>removeAll()</code>	Entfernt eine Sammlung von Elementen aus der Liste.	<code>boolean</code>
<code>removeIf()</code>	Entfernt alle Elemente aus der Liste, die eine angegebene Bedingung erfüllen.	<code>boolean</code>
<code>replaceAll()</code>	Ersetzt jedes Element in der Liste durch das Ergebnis einer Operation auf diesem Element.	<code>void</code>
<code>retainAll()</code>	Entfernt alle Elemente aus der Liste, die nicht zu einer angegebenen Sammlung gehören.	<code>boolean</code>
<code>set()</code>	Ersetzt ein Element an einer bestimmten Position in der Liste.	<code>T</code>
<code>size()</code>	Gibt die Anzahl der Elemente in der Liste zurück.	<code>int</code>
<code>sort()</code>	Sortiert die Liste.	<code>void</code>
<code>spliterator()</code>	Gibt ein Spliterator-Objekt für die ArrayList zurück.	<code>Spliterator</code>
<code>subList()</code>	Gibt eine Teilliste zurück, die Zugriff auf einen Bereich der Elemente dieser Liste bietet.	<code>List</code>
<code>toArray()</code>	Gibt ein Array zurück, das die Elemente der Liste enthält.	<code>Object[]</code>
<code>trimToSize()</code>	Reduziert die Kapazität der Liste auf die Anzahl der Elemente, falls erforderlich.	<code>void</code>

7.1 Beispielcode

```
1 import java.util.ArrayList;
2
3 public class ArrayListExample {
4     public static void main(String[] args) {
5         ArrayList<String> list = new ArrayList<>();
6         list.add("Apfel");
7         list.add("Banane");
8         System.out.println("Liste: " + list);
9         list.remove("Apfel");
10        System.out.println("Nach Entfernen: " + list);
11    }
12 }
```

8 Java LinkedList-Methoden

Methode	Beschreibung	Rückgabotyp
<code>add()</code>	Fügt ein Element zur Liste hinzu.	<code>boolean void</code>
<code>addAll()</code>	Fügt eine Sammlung von Elementen zur Liste hinzu.	<code>boolean</code>
<code>addFirst()</code>	Fügt ein Element an den Anfang der Liste hinzu.	<code>void</code>
<code>addLast()</code>	Fügt ein Element ans Ende der Liste hinzu.	<code>void</code>
<code>clear()</code>	Entfernt alle Elemente aus der Liste.	<code>void</code>
<code>clone()</code>	Erstellt eine Kopie der <code>LinkedList</code> .	<code>Object</code>
<code>contains()</code>	Prüft, ob ein Element in der Liste vorhanden ist.	<code>boolean</code>
<code>descendingIterator()</code>	Gibt einen Iterator zurück, um die Elemente der Liste in umgekehrter Reihenfolge zu durchlaufen.	<code>Iterator</code>
<code>element()</code>	Ruft das erste Element in der Liste ab. Ähnlich wie <code>getFirst()</code> .	<code>T</code>
<code>forEach()</code>	Führt eine Aktion für jedes Element in der Liste aus.	<code>void</code>
<code>get()</code>	Gibt das Element an einer bestimmten Position in der Liste zurück.	<code>T</code>
<code>getFirst()</code>	Gibt das erste Element in der Liste zurück.	<code>T</code>
<code>getLast()</code>	Gibt das letzte Element in der Liste zurück.	<code>T</code>
<code>indexOf()</code>	Gibt die Position des ersten Auftretens eines Elements in der Liste zurück.	<code>int</code>
<code>isEmpty()</code>	Prüft, ob die Liste leer ist.	<code>boolean</code>
<code>iterator()</code>	Gibt ein Iterator-Objekt für die <code>LinkedList</code> zurück.	<code>Iterator</code>
<code>lastIndexOf()</code>	Gibt die Position des letzten Auftretens eines Elements in der Liste zurück.	<code>int</code>
<code>listIterator()</code>	Gibt ein <code>ListIterator</code> -Objekt für die <code>LinkedList</code> zurück.	<code>ListIterator</code>
<code>offer()</code>	Fügt ein Element ans Ende der Liste hinzu.	<code>boolean</code>
<code>offerFirst()</code>	Fügt ein Element an den Anfang der Liste hinzu.	<code>boolean</code>
<code>offerLast()</code>	Fügt ein Element ans Ende der Liste hinzu.	<code>boolean</code>
<code>peek()</code>	Ruft das erste Element in der Liste ab. Ähnlich wie <code>getFirst()</code> .	<code>T</code>
<code>peekFirst()</code>	Ruft das erste Element in der Liste ab. Ähnlich wie <code>peek()</code> .	<code>T</code>
<code>peekLast()</code>	Ruft das letzte Element in der Liste ab.	<code>T</code>
<code>poll()</code>	Ruft das erste Element in der Liste ab und entfernt es.	<code>T</code>
<code>pollFirst()</code>	Ruft das erste Element in der Liste ab und entfernt es. Ähnlich wie <code>poll()</code> .	<code>T</code>
<code>pollLast()</code>	Ruft das letzte Element in der Liste ab und entfernt es.	<code>T</code>
<code>pop()</code>	Gibt das erste Element in der Liste zurück. Ähnlich wie <code>removeFirst()</code> .	<code>T</code>
<code>push()</code>	Fügt ein Element an den Anfang der Liste hinzu. Ähnlich wie <code>addFirst()</code> .	<code>void</code>
<code>remove()</code>	Entfernt ein Element aus der Liste.	<code>boolean T</code>
<code>removeAll()</code>	Entfernt eine Sammlung von Elementen aus der Liste.	<code>boolean</code>
<code>removeFirst()</code>	Entfernt das erste Element aus der Liste.	<code>T</code>
<code>removeFirstOccurrence()</code>	Entfernt das erste Auftreten eines angegebenen Elements in der Liste.	<code>boolean</code>
<code>removeIf()</code>	Entfernt alle Elemente aus der Liste, die eine angegebene Bedingung erfüllen.	<code>boolean</code>
<code>removeLast()</code>	Entfernt das letzte Element aus der Liste.	<code>T</code>
<code>removeLastOccurrence()</code>	Entfernt das letzte Auftreten eines angegebenen Elements in der Liste.	<code>boolean</code>
<code>replaceAll()</code>	Ersetzt jedes Element in der Liste mit dem Ergebnis einer Operation auf diesem Element.	<code>void</code>
<code>retainAll()</code>	Entfernt alle Elemente aus der Liste, die nicht zu einer angegebenen Sammlung gehören.	<code>boolean</code>
<code>set()</code>	Ersetzt ein Element an einer bestimmten Position in der Liste.	<code>T</code>
<code>size()</code>	Gibt die Anzahl der Elemente in der Liste zurück.	<code>int</code>
<code>sort()</code>	Sortiert die Liste.	<code>void</code>

Methode	Beschreibung	Rückgabebetyp
spliterator()	Gibt ein Spliterator-Objekt für die LinkedList zurück.	Spliterator
subList()	Gibt eine Teilliste zurück, die Zugriff auf einen Bereich der Elemente dieser Liste bietet.	List
toArray()	Gibt ein Array zurück, das die Elemente der Liste enthält.	Object[]

8.1 Beispielcode

```

1 import java.util.LinkedList;
2
3 public class LinkedListExample {
4     public static void main(String[] args) {
5         LinkedList<String> list = new LinkedList<>();
6         list.addFirst("Apfel");
7         list.addLast("Banane");
8         System.out.println("Liste: " + list);
9         list.removeFirst();
10        System.out.println("Nach Entfernen des Ersten: " + list);
11    }
12 }

```

9 Java HashMap-Methoden

Methoden	Beschreibung	Rückgabotyp
<code>clear()</code>	Entfernt alle Einträge aus der Map.	<code>void</code>
<code>clone()</code>	Erstellt eine Kopie der HashMap.	<code>Object</code>
<code>compute()</code>	Berechnet einen Wert für einen Eintrag basierend auf seinem Schlüssel und dem aktuellen Wert (falls vorhanden).	<code>V</code>
<code>computeIfAbsent()</code>	Berechnet einen Wert für einen Eintrag basierend auf seinem Schlüssel, nur wenn kein Eintrag mit dem Schlüssel existiert.	<code>V</code>
<code>computeIfPresent()</code>	Berechnet einen neuen Wert für einen Eintrag basierend auf seinem Schlüssel und aktuellen Wert, nur wenn ein Eintrag mit dem Schlüssel existiert.	<code>V</code>
<code>containsKey()</code>	Gibt an, ob ein Eintrag mit dem angegebenen Schlüssel in der Map existiert.	<code>boolean</code>
<code>containsValue()</code>	Gibt an, ob ein Eintrag mit dem angegebenen Wert in der Map existiert.	<code>boolean</code>
<code>entrySet()</code>	Gibt eine Menge aller Einträge in der Map zurück.	<code>Set<Map.Entry<K,V></code>
<code>forEach()</code>	Führt eine Aktion für jeden Eintrag in der Map aus.	<code>void</code>
<code>get()</code>	Gibt den Wert des Eintrags mit einem angegebenen Schlüssel zurück.	<code>V</code>
<code>getOrDefault()</code>	Gibt den Wert des Eintrags mit einem angegebenen Schlüssel oder einen Standardwert zurück, wenn der Eintrag nicht gefunden wird.	<code>V</code>
<code>isEmpty()</code>	Gibt an, ob die Map leer ist.	<code>boolean</code>
<code>keySet()</code>	Gibt eine Menge aller Schlüssel in der Map zurück.	<code>Set<K></code>
<code>merge()</code>	Berechnet einen Wert für einen Eintrag basierend auf seinem Schlüssel und Wert oder schreibt einen bestimmten Wert, wenn der Eintrag noch nicht existiert.	<code>V</code>
<code>put()</code>	Schreibt einen Eintrag in die Map.	<code>V</code>
<code>putAll()</code>	Schreibt alle Einträge aus einer anderen Map in diese.	<code>void</code>
<code>putIfAbsent()</code>	Schreibt einen Eintrag in die Map, aber nur, wenn kein Eintrag mit demselben Schlüssel existiert.	<code>V</code>
<code>remove()</code>	Entfernt einen Eintrag aus der Map.	<code>V boolean</code>
<code>replace()</code>	Schreibt in einen Eintrag in der Map, nur wenn er existiert.	<code>V boolean</code>
<code>replaceAll()</code>	Ersetzt den Wert jedes Eintrags mit dem Ergebnis einer Operation.	<code>void</code>
<code>size()</code>	Gibt die Anzahl der Einträge in der Map zurück.	<code>int</code>
<code>values()</code>	Gibt eine Sammlung zurück, die alle Werte in der Map enthält.	<code>Collection<V></code>

```

1      import java.util.HashMap;
2
3      public class HashMapExample {
4          public static void main(String[] args) {
5              HashMap<String, Integer> map = new HashMap<>();
6              map.put("Apfel", 10);
7              map.put("Banane", 20);
8              System.out.println("Map: " + map);
9              System.out.println("Wert fuer Apfel: " + map.get("Apfel"));
10         }
11     }

```

10 Java Scanner-Methoden

Methode	Beschreibung	Rückgabotyp
<code>close()</code>	Schließt das Scanner-Objekt.	<code>void</code>
<code>delimiter()</code>	Gibt das Trennzeichen zurück, das zum Trennen von Tokens verwendet wird.	<code>Pattern</code>
<code>findInLine()</code>	Gibt einen String zurück, der mit einem regulären Ausdruck in der aktuellen Zeile übereinstimmt.	<code>String</code>
<code>findWithinHorizon()</code>	Gibt einen String zurück, der mit einem regulären Ausdruck innerhalb einer angegebenen Anzahl von Zeichen übereinstimmt.	<code>String</code>
<code>hasNext()</code>	Gibt <code>true</code> zurück, wenn ein weiteres Token gefunden werden kann.	<code>boolean</code>
<code>hasNextBoolean()</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>boolean</code> -Wert darstellt.	<code>boolean</code>
<code>hasNextByte()</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>byte</code> -Wert darstellt.	<code>boolean</code>
<code>hasNextDouble()</code>	Gibt <code>true</code> zurück, wenn das nächste Token eine Zahl darstellt.	<code>boolean</code>
<code>hasNextFloat()</code>	Gibt <code>true</code> zurück, wenn das nächste Token eine Zahl darstellt.	<code>boolean</code>
<code>hasNextInt()</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>int</code> -Wert darstellt.	<code>boolean</code>
<code>hasNextLine()</code>	Gibt <code>true</code> zurück, wenn eine weitere Textzeile im Scanner verfügbar ist.	<code>boolean</code>
<code>hasNextLong()</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>long</code> -Wert darstellt.	<code>boolean</code>
<code>hasNextShort()</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>short</code> -Wert darstellt.	<code>boolean</code>
<code>locale()</code>	Gibt die Lokalisierung des Scanners zurück.	<code>Locale</code>
<code>next()</code>	Gibt das nächste Token im Scanner zurück.	<code>String</code>
<code>nextBoolean()</code>	Gibt den <code>boolean</code> -Wert des nächsten Tokens im Scanner zurück.	<code>boolean</code>
<code>nextByte()</code>	Gibt den <code>byte</code> -Wert des nächsten Tokens im Scanner zurück.	<code>byte</code>
<code>nextDouble()</code>	Gibt den <code>double</code> -Wert des nächsten Tokens im Scanner zurück.	<code>double</code>
<code>nextFloat()</code>	Gibt den <code>float</code> -Wert des nächsten Tokens im Scanner zurück.	<code>float</code>
<code>nextInt()</code>	Gibt den <code>int</code> -Wert des nächsten Tokens im Scanner zurück.	<code>int</code>
<code>nextLine()</code>	Gibt die nächste Textzeile im Scanner zurück.	<code>String</code>
<code>nextLong()</code>	Gibt den <code>long</code> -Wert des nächsten Tokens im Scanner zurück.	<code>long</code>
<code>nextShort()</code>	Gibt den <code>short</code> -Wert des nächsten Tokens im Scanner zurück.	<code>short</code>
<code>radix()</code>	Gibt die Basis des Scanners zurück.	<code>int</code>
<code>reset()</code>	Setzt die Konfiguration des Scanners zurück.	<code>Scanner</code>
<code>useDelimiter()</code>	Setzt das Trennzeichen, das vom Scanner zum Trennen von Tokens verwendet wird.	<code>Scanner</code>
<code>useLocale()</code>	Setzt die Lokalisierung, die vom Scanner verwendet wird.	<code>Scanner</code>
<code>useRadix()</code>	Setzt die Basis, die vom Scanner verwendet wird.	<code>Scanner</code>

10.1 Beispielcode

```

1 import java.util.Scanner;
2
3 public class ScannerExample {
4     public static void main(String[] args) {
5         Scanner scanner = new Scanner(System.in);
6         System.out.print("Geben Sie einen Namen ein: ");
7         String name = scanner.nextLine();
8         System.out.println("Eingegebener Name: " + name);
9         scanner.close();
10    }
11 }

```

11 Java File-Methoden

Methode	Beschreibung	Rückgabotyp
<code>createNewFile()</code>	Erstellt eine neue, leere Datei, wenn sie noch nicht existiert.	<code>boolean</code>
<code>delete()</code>	Löscht die Datei oder das Verzeichnis.	<code>boolean</code>
<code>exists()</code>	Prüft, ob die Datei oder das Verzeichnis existiert.	<code>boolean</code>
<code>canRead()</code>	Prüft, ob die Anwendung die Datei lesen kann.	<code>boolean</code>
<code>canWrite()</code>	Prüft, ob die Anwendung in die Datei schreiben kann.	<code>boolean</code>
<code>isFile()</code>	Gibt <code>true</code> zurück, wenn der Pfad eine Datei repräsentiert.	<code>boolean</code>
<code>isDirectory()</code>	Gibt <code>true</code> zurück, wenn der Pfad ein Verzeichnis repräsentiert.	<code>boolean</code>
<code>length()</code>	Gibt die Größe der Datei in Bytes zurück.	<code>long</code>
<code>getName()</code>	Gibt den Namen der Datei oder des Verzeichnisses zurück.	<code>String</code>
<code>getAbsolutePath()</code>	Gibt den absoluten Pfadnamen als <code>String</code> zurück.	<code>String</code>
<code>getPath()</code>	Gibt den Pfadnamen als <code>String</code> zurück.	<code>String</code>
<code>getParent()</code>	Gibt den Pfadnamen des übergeordneten Verzeichnisses als <code>String</code> zurück.	<code>String</code>
<code>list()</code>	Gibt ein Array von Namen der Dateien und Verzeichnisse in einem Verzeichnis zurück.	<code>String[]</code>
<code>listFiles()</code>	Gibt ein Array von <code>File</code> -Objekten in einem Verzeichnis zurück.	<code>File[]</code>
<code>mkdir()</code>	Erstellt ein Verzeichnis.	<code>boolean</code>
<code>mkdirs()</code>	Erstellt ein Verzeichnis und alle fehlenden übergeordneten Verzeichnisse.	<code>boolean</code>
<code>renameTo()</code>	Benennt die Datei oder das Verzeichnis um.	<code>boolean</code>
<code>setReadOnly()</code>	Markiert die Datei als nur lesbar.	<code>boolean</code>
<code>setWritable()</code>	Setzt die Schreibberechtigung.	<code>boolean</code>
<code>setReadable()</code>	Setzt die Leseberechtigung.	<code>boolean</code>
<code>setExecutable()</code>	Setzt die Ausführberechtigung.	<code>boolean</code>
<code>canExecute()</code>	Prüft, ob die Anwendung die Datei ausführen kann.	<code>boolean</code>

11.1 Beispielcode

```
1 import java.io.File;
2
3 public class FileExample {
4     public static void main(String[] args) {
5         File file = new File("example.txt");
6         if (file.exists()) {
7             System.out.println("Datei existiert: " + file.getName());
8         } else {
9             System.out.println("Datei existiert nicht.");
10        }
11    }
12 }
```


12 Java FileInputStream-Methoden

Methode	Beschreibung	Rückgabotyp
available()	Gibt die Anzahl der Bytes zurück, die ohne Blockierung gelesen werden können.	int
close()	Schließt den Stream und gibt Systemressourcen frei.	void
read()	Liest ein Byte an Daten.	int
read()	Liest Bytes in ein Array (oder einen Teil eines Arrays).	int
skip()	Überspringt und verwirft n Bytes an Daten.	long
getFD()	Gibt das mit dem Stream verbundene Dateideskriptor-Objekt zurück.	FileDescriptor
getChannel()	Gibt das eindeutige FileChannel-Objekt für diesen Stream zurück.	FileChannel

12.1 Beispielcode

```
1 import java.io.FileInputStream;
2
3 public class FileInputStreamExample {
4     public static void main(String[] args) throws Exception {
5         FileInputStream fis = new FileInputStream("example.txt");
6         int data = fis.read();
7         System.out.println("Gelesenes Byte: " + data);
8         fis.close();
9     }
10 }
```

13 Java FileOutputStream-Methoden

Methode	Beschreibung	Rückgabetyp
close()	Schließt den Stream und gibt Systemressourcen frei.	void
flush()	Leert den Stream – zwingt alle gepufferten Ausgabe-Bytes, sofort geschrieben zu werden.	void
write()	Schreibt ein Byte, ein Array von Bytes oder einen Teil eines Arrays in die Datei.	void
getFD()	Gibt das mit dem Stream verbundene Dateideskriptor-Objekt zurück.	FileDescriptor
getChannel()	Gibt das eindeutige FileChannel-Objekt für diesen Stream zurück.	FileChannel

13.1 Beispielcode

```
1 import java.io.FileOutputStream;
2
3 public class FileOutputStreamExample {
4     public static void main(String[] args) throws Exception {
5         FileOutputStream fos = new FileOutputStream("output.txt");
6         fos.write("Hallo".getBytes());
7         fos.flush();
8         fos.close();
9     }
10 }
```

14 Java BufferedReader-Methoden

Methode	Beschreibung	Rückgabebetyp
close()	Schließt den Stream und gibt Systemressourcen frei.	void
mark()	Markiert die aktuelle Position im Stream.	void
markSupported()	Prüft, ob mark() und reset() unterstützt werden.	boolean
read()	Liest ein einzelnes Zeichen.	int
read()	Liest Zeichen in ein Array oder einen Teil eines Arrays.	int
readLine()	Liest eine vollständige Textzeile.	String
ready()	Prüft, ob der Stream bereit zum Lesen ist.	boolean
reset()	Setzt den Stream auf die zuletzt markierte Position zurück.	void
skip()	Überspringt Zeichen im Stream.	long
lines()	Gibt einen Stream<String> aller Zeilen zurück.	Stream<String>
transferTo()	Liest alle Zeichen und schreibt sie in den angegebenen Writer.	long

14.1 Beispielcode

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3
4 public class BufferedReaderExample {
5     public static void main(String[] args) throws Exception {
6         BufferedReader br = new BufferedReader(new FileReader("example.txt"));
7         String line = br.readLine();
8         System.out.println("Gelesene Zeile: " + line);
9         br.close();
10    }
11 }
```

15 Java BufferedWriter-Methoden

Methode	Beschreibung	Rückgabebetyp
<code>close()</code>	Schließt den Stream und leert ihn zuerst.	<code>void</code>
<code>flush()</code>	Zwingt alle gepufferten Ausgabe-Bytes, sofort geschrieben zu werden.	<code>void</code>
<code>newLine()</code>	Schreibt den systemabhängigen Zeilentrenner.	<code>void</code>
<code>write()</code>	Schreibt ein einzelnes Zeichen, ein Array, einen Teil eines Arrays, einen String oder einen Teil eines Strings.	<code>void</code>
<code>append()</code>	Fügt Zeichen oder Zeichenfolgen an den Writer an.	<code>Writer</code>

15.1 Beispielcode

```
1 import java.io.BufferedWriter;
2 import java.io.FileWriter;
3
4 public class BufferedWriterExample {
5     public static void main(String[] args) throws Exception {
6         BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));
7         bw.write("Hallo Welt");
8         bw.newLine();
9         bw.flush();
10        bw.close();
11    }
12 }
```

16 Java Iterator-Methoden

Methode	Beschreibung	Rückgabotyp
<code>hasNext()</code>	Gibt <code>true</code> zurück, wenn es weitere Elemente in der Iteration gibt.	<code>boolean</code>
<code>next()</code>	Gibt das nächste Element in der Iteration zurück.	<code>T</code>
<code>remove()</code>	Entfernt das letzte von <code>next()</code> zurückgegebene Element.	<code>void</code>

16.1 Beispielcode

```
1 import java.util.ArrayList;
2 import java.util.Iterator;
3
4 public class IteratorExample {
5     public static void main(String[] args) {
6         ArrayList<String> list = new ArrayList<>();
7         list.add("Apfel");
8         list.add("Banane");
9         Iterator<String> it = list.iterator();
10        while (it.hasNext()) {
11            System.out.println(it.next());
12        }
13    }
14 }
```

17 Java Collections-Methoden

Methode	Beschreibung	Rückgabotyp
<code>sort()</code>	Sortiert die Liste in aufsteigender Reihenfolge (nach natürlicher Ordnung).	<code>void</code>
<code>binarySearch()</code>	Sucht nach einem Schlüssel in einer sortierten Liste und gibt dessen Index zurück (sonst < 0).	<code>int</code>
<code>reverse()</code>	Kehrt die Reihenfolge der Elemente in einer Liste um.	<code>void</code>
<code>reverseOrder()</code>	Gibt einen Komparator zurück, der die umgekehrte natürliche Ordnung erzwingt.	<code>Comparator</code>
<code>shuffle()</code>	Permutiert die Elemente einer Liste zufällig.	<code>void</code>
<code>swap()</code>	Tauscht die Elemente an den angegebenen Positionen in einer Liste.	<code>void</code>
<code>frequency()</code>	Gibt die Anzahl der Elemente zurück, die gleich dem angegebenen Objekt sind.	<code>int</code>
<code>max()</code>	Gibt das maximale Element einer Sammlung zurück (nach natürlicher Ordnung).	<code>T</code>
<code>min()</code>	Gibt das minimale Element einer Sammlung zurück (nach natürlicher Ordnung).	<code>T</code>
<code>disjoint()</code>	Prüft, ob zwei Sammlungen keine gemeinsamen Elemente haben.	<code>boolean</code>
<code>copy()</code>	Kopiert alle Elemente aus einer Liste in eine andere.	<code>void</code>
<code>fill()</code>	Ersetzt alle Elemente der Liste durch das angegebene Objekt.	<code>void</code>
<code>replaceAll()</code>	Ersetzt alle Vorkommen eines Werts durch einen anderen in einer Liste.	<code>boolean</code>
<code>unmodifiableList()</code>	Gibt eine unveränderbare (nur lesbare) Ansicht der angegebenen Liste zurück.	<code>List</code>
<code>unmodifiableSet()</code>	Gibt eine unveränderbare (nur lesbare) Ansicht der angegebenen Menge zurück.	<code>Set</code>
<code>unmodifiableMap()</code>	Gibt eine unveränderbare (nur lesbare) Ansicht der angegebenen Map zurück.	<code>Map</code>

17.1 Beispielcode

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3
4 public class CollectionsExample {
5     public static void main(String[] args) {
6         ArrayList<Integer> numbers = new ArrayList<>();
7         numbers.add(5);
8         numbers.add(2);
9         numbers.add(8);
10        Collections.sort(numbers);
11        System.out.println("Sortierte Liste: " + numbers);
12    }
13 }
```

18 Java System-Methoden

Methode	Beschreibung	Rückgabotyp
<code>currentTimeMillis()</code>	Gibt die aktuelle Zeit in Millisekunden seit dem 1. Januar 1970 (Unix-Epoche) zurück.	<code>long</code>
<code>nanoTime()</code>	Gibt den aktuellen Wert des präzisesten verfügbaren Systemtimers in Nanosekunden zurück.	<code>long</code>
<code>exit()</code>	Beendet das laufende Java-Programm. Ein Status ungleich Null deutet auf eine anormale Beendigung hin.	<code>void</code>
<code>gc()</code>	Schlägt vor, dass die Java Virtual Machine den Garbage Collector ausführt.	<code>void</code>
<code>getProperty()</code>	Ruft die Systemeigenschaft ab, die durch den Schlüssel angegeben ist, oder gibt den Standardwert zurück, wenn sie nicht gefunden wird.	<code>String</code>
<code>getProperties()</code>	Gibt alle Systemeigenschaften als <code>Properties</code> -Objekt zurück.	<code>Properties</code>
<code>setProperty()</code>	Setzt die Systemeigenschaft, die durch den Schlüssel angegeben ist.	<code>String</code>
<code>clearProperty()</code>	Entfernt die Systemeigenschaft, die durch den Schlüssel angegeben ist.	<code>String</code>
<code>arraycopy()</code>	Kopiert Elemente aus dem Quell-Array in das Ziel-Array.	<code>void</code>
<code>identityHashCode()</code>	Gibt den Standard-Hash-Code des gegebenen Objekts zurück (auch wenn <code>hashCode()</code> überschrieben wurde).	<code>int</code>
<code>lineSeparator()</code>	Gibt den systemabhängigen Zeilentrenner-String zurück (z. B. „\n“ unter Linux, „\r\n“ unter Windows).	<code>String</code>
<code>getenv()</code>	Gibt eine unveränderbare Map der Systemumgebungsvariablen zurück.	<code>Map<String,String></code>
<code>getenv()</code>	Gibt den Wert der angegebenen Umgebungsvariablen zurück oder <code>null</code> , wenn sie nicht existiert.	<code>String</code>

18.1 Beispielcode

```
1 public class SystemExample {
2     public static void main(String[] args) {
3         System.out.println("Aktuelle Zeit (ms): " + System.currentTimeMillis());
4         System.setProperty("myProperty", "myValue");
5         System.out.println("Eigenschaft: " + System.getProperty("myProperty"));
6     }
7 }
```

19 Java Fehler und Ausnahmen

Fehler/Ausnahme	Beschreibung
ArithmeticError	Tritt auf, wenn eine numerische Berechnung fehlschlägt.
ArrayIndexOutOfBoundsException	Tritt auf, wenn versucht wird, auf einen nicht existierenden Index in einem Array zuzugreifen.
ClassFormatError	Tritt auf, wenn eine Klassendatei nicht zugänglich ist.
ClassNotFoundException	Tritt auf, wenn versucht wird, auf eine nicht existierende Klasse zuzugreifen.
ConcurrentModificationException	Tritt auf, wenn ein Element aus einer iterierbaren Struktur hinzugefügt oder entfernt wird.
FileNotFoundException	Tritt auf, wenn eine Datei nicht zugänglich ist.
IncompatibleClassChangeError	Tritt auf, wenn eine Änderung in einer Basisklasse vorgenommen wurde, nachdem eine Unterklasse bereits initialisiert wurde.
InputMismatchException	Tritt auf, wenn eine falsche Eingabe erfolgt (z. B. Text bei einer numerischen Eingabe).
InterruptedException	Tritt auf, wenn ein Thread unterbrochen wird, während er wartet oder schläft.
InvalidClassException	Tritt auf, wenn die Serialisierung ein Problem mit einer Klasse feststellt.
IOException	Tritt auf, wenn eine Ein-/Ausgabeoperation fehlschlägt.
NegativeArraySizeException	Tritt auf, wenn versucht wird, ein Array mit negativer Größe zu erstellen.
NoClassDefFoundError	Tritt auf, wenn die Klasse zur Laufzeit nicht gefunden wird.
NoSuchFieldException	Tritt auf, wenn versucht wird, auf ein nicht existierendes Klassenfeld zuzugreifen.
NoSuchMethodException	Tritt auf, wenn versucht wird, auf eine nicht existierende Klassenmethode zuzugreifen.
NullPointerException	Tritt auf, wenn versucht wird, auf eine Objektreferenz zuzugreifen, die null ist.
NumberFormatException	Tritt auf, wenn ein angegebener String nicht in einen numerischen Typ umgewandelt werden kann.
RuntimeException	Tritt auf, wenn eine Ausnahme zur Laufzeit auftritt.
StringIndexOutOfBoundsException	Tritt auf, wenn versucht wird, auf ein Zeichen in einem String zuzugreifen, das nicht existiert.
TypeNotPresentException	Tritt auf, wenn ein Typ nicht gefunden werden kann.
IllegalArgumentException	Tritt auf, wenn ein ungültiges Argument an eine Methode übergeben wird.
IllegalStateException	Tritt auf, wenn eine Methode zu einem ungültigen Zeitpunkt aufgerufen wird.

19.1 Beispielcode

```
1 public class ExceptionExample {
2     public static void main(String[] args) {
3         try {
4             int[] arr = new int[5];
5             System.out.println(arr[10]); // ArrayIndexOutOfBoundsException
6         } catch (ArrayIndexOutOfBoundsException e) {
7             System.out.println("Fehler: " + e.getMessage());
8         }
9     }
10 }
```

20 Java-Datum

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>adjustInto(Temporal temporal)</code>	Temporal	Passt ein temporales Objekt an dieses Datum an.	<pre> 1 LocalDateTime ldt = LocalDateTime.now(); 2 LocalDate d = LocalDate.now() .adjustInto(ldt); </pre>
<code>atStartOfDay()</code>	LocalDateTime	Datum + 00:00:00.	<pre> 1 LocalDateTime start = LocalDate.now(). atStartOfDay(); </pre>
<code>atStartOfDay(ZoneId zone)</code>	ZonedDateTime	Früheste gültige Zeit in Zeitzone.	<pre> 1 ZonedDateTime zdt = LocalDate.now() 2 .atStartOfDay(ZoneId.of("Europe/Berlin")); </pre>
<code>atTime(int hour, int minute)</code>	LocalDateTime	Kombiniert mit Stunde/Minute.	<pre> 1 LocalDateTime dt = LocalDate.now().atTime (14, 30); </pre>
<code>atTime(int hour, int minute, int second)</code>	LocalDateTime	Mit Sekunden.	<pre> 1 LocalDateTime dt = LocalDate.now().atTime (14, 30, 0); </pre>
<code>atTime(int hour, int minute, int second, int nanoOfSecond)</code>	LocalDateTime	Mit Nanosekunden.	<pre> 1 LocalDateTime dt = LocalDate.now() 2 .atTime(14, 30, 0, 500 _000_000); </pre>
<code>atTime(LocalTime time)</code>	LocalDateTime	Mit LocalTime.	<pre> 1 LocalDateTime dt = LocalDate.now().atTime(LocalTime.NOON); </pre>
<code>atTime(OffsetTime time)</code>	OffsetDateTime	Mit OffsetTime.	<pre> 1 OffsetTime ot = OffsetTime. of(LocalTime.NOON, ZoneOffset.UTC); 2 OffsetDateTime odt = LocalDate.now().atTime(ot); </pre>
<code>compareTo(ChronoLocalDate other)</code>	int	Vergleicht zwei Daten.	<pre> 1 int cmp = LocalDate.now(). compareTo(LocalDate.of (2000,1,1)); </pre>
<code>datesUntil(LocalDate endExclusive)</code>	Stream<LocalDate>	Stream bis exklusiv.	<pre> 1 long cnt = LocalDate.of (2025,1,1) 2 .datesUntil(LocalDate.of (2025,1,10)).count (); // 9 </pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
datesUntil(LocalDate endExclusive, Period step)	Stream<LocalDate>	Mit Schritt (z. B. monatlich).	<pre> 1 LocalDate.now() 2 .datesUntil(LocalDate.now() .plusYears(1), Period.ofMonths(2)) 3 .forEach(System.out::println); </pre>
equals(Object obj)	boolean	Prüft Gleichheit.	<pre> 1 boolean eq = LocalDate.now() .equals(LocalDate.now()); // true </pre>
format(DateTimeFormatter formatter)	String	Formatiert mit Formatter.	<pre> 1 String s = LocalDate.now() 2 .format(DateTimeFormatter.ofPattern("dd.MM. yyyy")); </pre>
from(TemporalAccessor temporal)	static LocalDate	Aus temporalem Objekt.	<pre> 1 LocalDate d = LocalDate. from(LocalDateTime.now()); </pre>
get(TemporalField field)	int	Wert eines Feldes.	<pre> 1 int day = LocalDate.now(). get(ChronoField. DAY_OF_MONTH); </pre>
getChronology()	IsoChronology	Gibt ISO-Chronologie.	<pre> 1 IsoChronology c = LocalDate. now().getChronology(); </pre>
getDayOfMonth()	int	Tag im Monat.	<pre> 1 int tag = LocalDate.now(). getDayOfMonth(); </pre>
getDayOfWeek()	DayOfWeek	Wochentag.	<pre> 1 DayOfWeek dow = LocalDate. now().getDayOfWeek(); </pre>
getDayOfYear()	int	Tag im Jahr.	<pre> 1 int doy = LocalDate.now(). getDayOfYear(); </pre>
getEra()	IsoEra	Ära (CE/BCE).	<pre> 1 IsoEra era = LocalDate.now() .getEra(); </pre>
getLong(TemporalField field)	long	Feld als long.	<pre> 1 long year = LocalDate.now() .getLong(ChronoField. YEAR); </pre>
getMonth()	Month	Monat als Enum.	<pre> 1 Month m = LocalDate.now(). getMonth(); </pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
getMonthValue()	int	Monat (1–12).	<pre>int mv = LocalDate.now().getMonthValue();</pre>
getYear()	int	Jahr.	<pre>int y = LocalDate.now().getYear();</pre>
hashCode()	int	Hash-Code.	<pre>int h = LocalDate.now().hashCode();</pre>
isAfter(ChronoLocalDate other)	boolean	Ist nach?	<pre>boolean after = LocalDate.now().isAfter(LocalDate.of(2000,1,1));</pre>
isBefore(ChronoLocalDate other)	boolean	Ist vor?	<pre>boolean before = LocalDate.of(2000,1,1).isBefore(LocalDate.now());</pre>
isEqual(ChronoLocalDate other)	boolean	Ist gleich?	<pre>boolean eq = LocalDate.now().isEqual(LocalDate.now());</pre>
isLeapYear()	boolean	Schaltjahr?	<pre>boolean leap = LocalDate.of(2024,1,1).isLeapYear(); // true</pre>
isSupported(TemporalField field)	boolean	Feld unterstützt?	<pre>boolean sup = LocalDate.now().isSupported(ChronoField.HOUR_OF_DAY); // false</pre>
isSupported(TemporalUnit unit)	boolean	Einheit unterstützt?	<pre>boolean sup = LocalDate.now().isSupported(ChronoUnit.DAYS); // true</pre>
lengthOfMonth()	int	Tage im Monat.	<pre>int days = LocalDate.of(2025,2,1).lengthOfMonth(); // 28</pre>
lengthOfYear()	int	Tage im Jahr.	<pre>int days = LocalDate.of(2024,1,1).lengthOfYear(); // 366</pre>
minus(long amount, TemporalUnit unit)	LocalDate	Subtrahiert Einheit.	<pre>LocalDate past = LocalDate.now().minus(5, ChronoUnit.MONTHS);</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
minus(TemporalAmount amount)	LocalDate	Subtrahiert Period.	<pre>LocalDate past = LocalDate.now().minus(Period.ofYears(1));</pre>
minusDays(long days)	LocalDate	Subtrahiert Tage.	<pre>LocalDate ago = LocalDate.now().minusDays(7);</pre>
minusMonths(long months)	LocalDate	Subtrahiert Monate.	<pre>LocalDate m = LocalDate.now().minusMonths(3);</pre>
minusWeeks(long weeks)	LocalDate	Subtrahiert Wochen.	<pre>LocalDate w = LocalDate.now().minusWeeks(2);</pre>
minusYears(long years)	LocalDate	Subtrahiert Jahre.	<pre>LocalDate y = LocalDate.now().minusYears(10);</pre>
now()	static LocalDate	Aktuelles Datum.	<pre>LocalDate today = LocalDate.now();</pre>
now(Clock clock)	static LocalDate	Mit Clock.	<pre>LocalDate now = LocalDate.now(Clock.systemUTC());</pre>
now(ZoneId zone)	static LocalDate	In Zeitzone.	<pre>LocalDate berlin = LocalDate.now(ZoneId.of("Europe/Berlin"));</pre>
of(int year, int month, int day)	static LocalDate	Aus Jahr/Monat/Tag.	<pre>LocalDate d = LocalDate.of(2025, 12, 25);</pre>
of(int year, Month month, int day)	static LocalDate	Mit Month.	<pre>LocalDate d = LocalDate.of(2025, Month.DECEMBER, 25);</pre>
ofEpochDay(long epochDay)	static LocalDate	Aus Epoch-Tag.	<pre>LocalDate d = LocalDate.ofEpochDay(0); // 1970-01-01</pre>
ofInstant(Instant instant, ZoneId zone)	static LocalDate	Aus Instant.	<pre>LocalDate d = LocalDate.ofInstant(Instant.now(), ZoneId.of("UTC"));</pre>
ofYearDay(int year, int dayOfYear)	static LocalDate	Aus Jahr + Tag im Jahr.	<pre>LocalDate d = LocalDate.ofYearDay(2025, 100);</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>parse(CharSequence text)</code>	static LocalDate	Aus YYYY-MM-DD.	<pre>1 LocalDate d = LocalDate. parse("2025-12-25");</pre>
<code>parse(CharSequence text, DateTimeFormatter fmt)</code>	static LocalDate	Mit Formatter.	<pre>1 DateTimeFormatter f = DateTimeFormatter. ofPattern("dd.MM.yyyy") ; 2 LocalDate d = LocalDate. parse("25.12.2025", f);</pre>
<code>plus(long amount, TemporalUnit unit)</code>	LocalDate	Addiert Einheit.	<pre>1 LocalDate future = LocalDate.now().plus(3, ChronoUnit.MONTHS);</pre>
<code>plus(TemporalAmount amount)</code>	LocalDate	Addiert Period.	<pre>1 LocalDate f = LocalDate.now ().plus(Period.ofDays (10));</pre>
<code>plusDays(long days)</code>	LocalDate	Addiert Tage.	<pre>1 LocalDate d = LocalDate.now ().plusDays(5);</pre>
<code>plusMonths(long months)</code>	LocalDate	Addiert Monate.	<pre>1 LocalDate m = LocalDate.now ().plusMonths(1);</pre>
<code>plusWeeks(long weeks)</code>	LocalDate	Addiert Wochen.	<pre>1 LocalDate w = LocalDate.now ().plusWeeks(1);</pre>
<code>plusYears(long years)</code>	LocalDate	Addiert Jahre.	<pre>1 LocalDate y = LocalDate.now ().plusYears(1);</pre>
<code>query(TemporalQuery<R> query)</code>	<R> R	Benutzerdefinierte Abfrage.	<pre>1 Year y = LocalDate.now() 2 .query(TemporalQueries. localDate().andThen (TemporalQueries:: year));</pre>
<code>range(TemporalField field)</code>	ValueRange	Gültiger Wertebereich.	<pre>1 ValueRange r = LocalDate. now().range(ChronoField .DAY_OF_MONTH);</pre>
<code>toEpochSecond(LocalTime time, ZoneOffset offset)</code>	long	Sekunden seit 1970.	<pre>1 long sec = LocalDate.now(). toEpochSecond(LocalTime .NOON, ZoneOffset.UTC);</pre>
<code>toString()</code>	String	YYYY-MM-DD.	<pre>1 String s = LocalDate.now(). toString();</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>until(ChronoLocalDate endExclusive)</code>	Period	Period zwischen Daten.	<pre>Period p = LocalDate.of (2000,1,1).until(LocalDate.now());</pre>
<code>until(Temporal endExclusive, TemporalUnit unit)</code>	long	Differenz in Einheit.	<pre>long days = LocalDate.now() .until(LocalDate.of (2025,12,25), ChronoUnit.DAYS);</pre>
<code>with(TemporalAdjuster adjuster)</code>	LocalDate	Mit Adjuster.	<pre>LocalDate first = LocalDate .now().with(TemporalAdjusters. firstDayOfMonth());</pre>
<code>with(TemporalField field, long value)</code>	LocalDate	Setzt Feld.	<pre>LocalDate d = LocalDate.now ().with(ChronoField. DAY_OF_MONTH, 15);</pre>
<code>withDayOfMonth(int dayOfMonth)</code>	LocalDate	Ändert Tag im Monat.	<pre>LocalDate d = LocalDate.now ().withDayOfMonth(1);</pre>
<code>withDayOfYear(int dayOfYear)</code>	LocalDate	Ändert Tag im Jahr.	<pre>LocalDate d = LocalDate.now ().withDayOfYear(200);</pre>
<code>withMonth(int month)</code>	LocalDate	Ändert Monat.	<pre>LocalDate d = LocalDate.now ().withMonth(12);</pre>
<code>withYear(int year)</code>	LocalDate	Ändert Jahr.	<pre>LocalDate d = LocalDate.now ().withYear(2030);</pre>